Trade and Portfolio Analysis Database System

Database Design, Implementation, and Trade Analytics

Group 173 Yanggao
Yingqi Tan Zhesheng Xu Hao Yang

MISY330 Database Design and Implementation
Spring 2026

Business Overview

- This project models a trade and portfolio analysis business that tracks stocks, exchanges, market data, strategies, factors, and trading records.
- A database is needed because trade analysis depends on structured relationships across companies, exchanges, portfolios, tax rules, and transaction history.
- The system supports practical tasks such as tax calculation, FIFO cost matching, factor-based stock screening, realized profit monitoring, and cross-market comparison.
- Business benefits include consistent record keeping, reusable query logic, faster reporting, and better decision support for portfolio management.

Database Scope and Main Entities

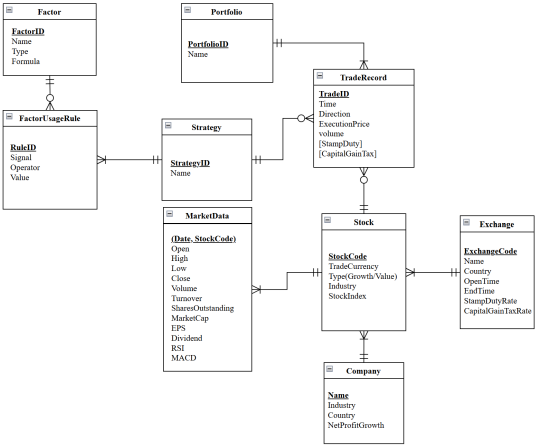
- The database covers the full workflow from reference data to trade reporting.
- `Company`, `Exchange`, and `Stock` define the market structure.
- `MarketData` stores daily valuation and indicator information.
- `Portfolio` and `TradeRecord` capture holdings and executed transactions.
- `Strategy` stores investment approaches used for trades.
- `Factor` stores screening dimensions such as PE ratio, dividend yield, RSI, and MACD.
- `FactorUsageRule` links factors to strategies through buy/sell rules.
- Together, these tables support tax analysis, FIFO matching, stock screening, and realized P&L reporting.

How the System Works

- The database is built in layers so each part of the business process has a clear role.
- **Reference layer:** `Company` stores issuer fundamentals, `Exchange` stores market rules and tax rates, and `Stock` connects each tradable ticker to its company and exchange.
- **Market layer:** `MarketData` stores date-based observations such as close price, EPS, dividend, RSI, and MACD for later valuation and factor analysis.
- **Decision layer:** `Factor`, `Strategy`, and `FactorUsageRule` model how screening rules are translated into buy and sell signals.
- **Transaction layer:** `Portfolio` identifies the investment account, and `TradeRecord` stores the executed trades with quantity, price, tax fields, selected strategy, and linked stock.
- **Analysis layer:** SQL queries combine these tables to calculate taxes, apply FIFO logic, summarize realized P&L, and screen securities for investment decisions.

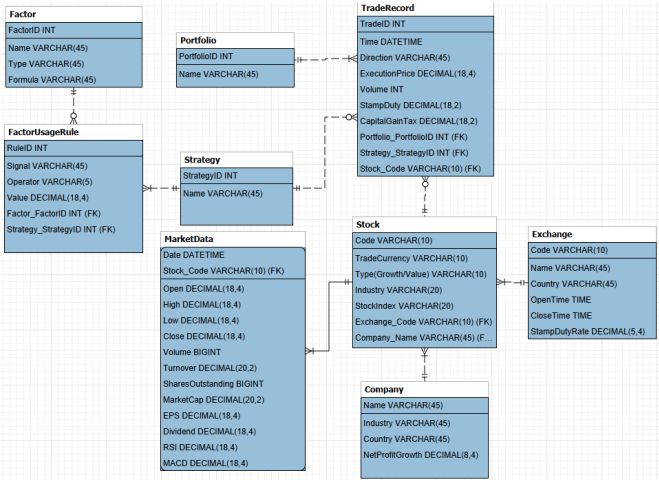
ER Diagram

- The ER diagram shows 9 entities, which satisfies the requirement of at least 8 entities.
- Primary keys identify each table, and foreign keys connect the business workflow from reference data to transactions.
- Major relationships include Exchange–Stock, Company–Stock, Stock–MarketData, Portfolio–TradeRecord, and Strategy–FactorUsageRule.
- `TradeRecord` is the central transaction table because each trade belongs to one portfolio, one strategy, and one stock.



Logical Data Model

- The logical model translates the ER design into implementation-ready tables and relationship directions.
- `FactorUsageRule` is the key associative table that resolves the many-to-many relationship between `Strategy` and `Factor`.
- `MarketData` uses a composite key of date and stock code to store time-series market observations.
- Optional tax fields in `TradeRecord` support later update queries for stamp duty and capital gain tax.



Database Implementation

- The schema was implemented in MySQL Workbench, and `GroupProject.sql` creates the database, tables, keys, and sample data.
- Parent tables are created first, then dependent tables such as `Stock`, `MarketData`, `FactorUsageRule`, and `TradeRecord`.
- The script loads sample records and keeps PK/FK links consistent so later tax, FIFO, and reporting queries can run directly on the same dataset.

Algorithm: `Create database`

```
Create parent tables and PK/FK rules
Insert lookup data
Load dependent tables

Run update and analysis queries
```

Query 1 SQL: FIFO Cost Basis Check

```sql
WITH TradeFlow AS (
    SELECT
        tr.*,
        SUM(CASE WHEN tr.Direction = 'Buy' THEN tr.Volume ELSE 0 END)
            OVER (
                PARTITION BY tr.Stock_Code, tr.Portfolio_PortfolioID
                ORDER BY tr.Time ASC, tr.TradeID ASC
            ) AS Cum_Buy,
        SUM(CASE WHEN tr.Direction = 'Sell' THEN tr.Volume ELSE 0 END)
            OVER (
                PARTITION BY tr.Stock_Code, tr.Portfolio_PortfolioID
                ORDER BY tr.Time ASC, tr.TradeID ASC
            ) AS Cum_Sell
    FROM TradeRecord tr
)
SELECT
    tf.TradeID,
    tf.Portfolio_PortfolioID,
    tf.Stock_Code,
    tf.Direction,
    tf.Volume,
    tf.ExecutionPrice,
    CASE
        WHEN tf.Direction = 'Sell' THEN (
            SELECT tr_b.ExecutionPrice
            FROM TradeRecord tr_b
            WHERE tr_b.Stock_Code = tf.Stock_Code
              AND tr_b.Portfolio_PortfolioID = tf.Portfolio_PortfolioID
              AND tr_b.Direction = 'Buy'
              AND (
                    SELECT SUM(tr_x.Volume)
                    FROM TradeRecord tr_x
                    WHERE tr_x.Stock_Code = tf.Stock_Code
                      AND tr_x.Portfolio_PortfolioID = tf.Portfolio_PortfolioID
                      AND tr_x.Direction = 'Buy'
                      AND (
                            tr_x.Time < tr_b.Time
                            OR (tr_x.Time = tr_b.Time AND tr_x.TradeID <= tr_b.TradeID)
                          )
                  ) >= tf.Cum_Sell
            ORDER BY tr_b.Time ASC, tr_b.TradeID ASC
            LIMIT 1
        )
        ELSE NULL
    END AS FIFO_Cost_Basis
FROM TradeFlow tf;
```

Query 1 Result: FIFO Cost Basis Check

- FIFO matches each sell trade to the earliest eligible buy in the same stock and portfolio.
- NVDA sells match the expected buy prices, while trade 20 has no prior buy match.

TradeID	Portfolio_PortfolioID	Stock_Code	Direction	Volume	ExecutionPrice	FIFO_Cost_Basis
4	3	002594	Buy	5000	102.6500	NULL
7	3	002594	Buy	3000	102.4900	NULL
14	3	002594	Sell	4000	101.6500	102.6500
5	4	00700	Buy	400	496.6000	NULL
15	4	00700	Sell	200	489.2000	496.6000
9	4	00941	Buy	2000	79.4500	NULL
16	4	00941	Sell	1000	80.0500	79.4500
20	3	01211	Sell	1000	103.9000	NULL
6	5	601988	Buy	10000	5.9400	NULL
17	5	601988	Sell	5000	5.9400	5.9400
1	1	AAPL	Buy	1000	256.4300	NULL
13	1	AAPL	Sell	600	251.9000	256.4300
3	2	COST	Buy	50	996.2100	NULL
12	2	COST	Sell	30	1014.9600	996.2100
10	2	JPM	Buy	200	292.2200	NULL
19	2	JPM	Sell	100	294.6000	292.2200
2	1	NVDA	Buy	500	175.8600	NULL
8	1	NVDA	Buy	400	177.3900	NULL
11	1	NVDA	Sell	500	177.3900	175.8600
18	1	NVDA	Sell	200	177.3900	177.3900

Query 2 SQL (Part 1): Stamp Duty Update

```sql
UPDATE TradeRecord tr
JOIN Stock s ON tr.Stock_Code = s.Code
JOIN Exchange e ON s.Exchange_Code = e.Code
SET tr.StampDuty = ROUND(
    CASE
        -- HK stocks: both buy and sell are charged
        WHEN e.Code = 'HKEX'
            THEN tr.ExecutionPrice * tr.Volume * e.StampDutyRate

        -- A-shares: only sell is charged
        WHEN e.Code IN ('SSE', 'SZSE') AND tr.Direction = 'Sell'
            THEN tr.ExecutionPrice * tr.Volume * e.StampDutyRate

        -- US stocks or A-share buys
        ELSE 0
    END
, 2);
```

Query 2 SQL (Part 2): Capital Gain Tax Update

```sql
SET SQL_SAFE_UPDATES = 0;

UPDATE TradeRecord tr
JOIN (
    SELECT
        t_all.TradeID,
        (
            SELECT t_b.ExecutionPrice
            FROM TradeRecord t_b
            WHERE t_b.Stock_Code = t_all.Stock_Code
              AND t_b.Portfolio_PortfolioID = t_all.Portfolio_PortfolioID
              AND t_b.Direction = 'Buy'
              AND (
                    SELECT SUM(tb2.Volume)
                    FROM TradeRecord tb2
                    WHERE tb2.Direction = 'Buy'
                      AND tb2.Stock_Code = t_all.Stock_Code
                      AND tb2.Portfolio_PortfolioID = t_all.Portfolio_PortfolioID
                      AND (
                            tb2.Time < t_b.Time
                            OR (tb2.Time = t_b.Time AND tb2.TradeID <= t_b.TradeID)
                          )
                  ) >= (
                    SELECT SUM(ts2.Volume)
                    FROM TradeRecord ts2
                    WHERE ts2.Direction = 'Sell'
                      AND ts2.Stock_Code = t_all.Stock_Code
                      AND ts2.Portfolio_PortfolioID = t_all.Portfolio_PortfolioID
                      AND (
                            ts2.Time < t_all.Time
                            OR (ts2.Time = t_all.Time AND ts2.TradeID <= t_all.TradeID)
                          )
                  )
            ORDER BY t_b.Time ASC, t_b.TradeID ASC
            LIMIT 1
        ) AS FIFO_Cost
    FROM TradeRecord t_all
    WHERE t_all.Direction = 'Sell'
) AS fifo ON tr.TradeID = fifo.TradeID
JOIN Stock s ON tr.Stock_Code = s.Code
JOIN Exchange e ON s.Exchange_Code = e.Code
SET tr.CapitalGainTax = ROUND(
    GREATEST(tr.ExecutionPrice - fifo.FIFO_Cost, 0) * tr.Volume * e.CapitalGainTaxRate,
    2
)
WHERE e.CapitalGainTaxRate > 0;
```

Query 2 SQL (Part 3): Tax Result Check

```sql
SELECT
    tr.TradeID,
    tr.Stock_Code,
    tr.Direction,
    e.Code AS Exch,
    tr.ExecutionPrice,
    tr.Volume,
    tr.StampDuty,
    tr.CapitalGainTax
FROM TradeRecord tr
JOIN Stock s ON tr.Stock_Code = s.Code
JOIN Exchange e ON s.Exchange_Code = e.Code
ORDER BY tr.Time ASC, tr.TradeID ASC;
```

Query 2 Result: Updated Tax Results by Exchange

- Tax fields are updated by joining each trade to the exchange-specific stamp duty and capital gain tax rules.
- HKEX trades show stamp duty, while profitable US sells such as NVDA and COST show capital gain tax.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TradeID	Stock_Code	Direction	Exch	ExecutionPrice	Volume	StampDuty	CapitalGainTax
1	AAPL	Buy	NASDAQ	256.4300	1000	0.0000	0.0000
2	NVDA	Buy	NASDAQ	175.8600	500	0.0000	0.0000
3	COST	Buy	NASDAQ	996.2100	50	0.0000	0.0000
4	002594	Buy	SZSE	102.6500	5000	0.0000	0.0000
5	00700	Buy	HKEX	496.6000	400	198.6400	0.0000
6	601988	Buy	SSE	5.9400	10000	0.0000	0.0000
7	002594	Buy	SZSE	102.4900	3000	0.0000	0.0000
8	NVDA	Buy	NASDAQ	177.3900	400	0.0000	0.0000
9	00941	Buy	HKEX	79.4500	2000	158.9000	0.0000
10	JPM	Buy	NYSE	292.2200	200	0.0000	0.0000
11	NVDA	Sell	NASDAQ	177.3900	500	0.0000	114.7500
12	COST	Sell	NASDAQ	1014.9600	30	0.0000	84.3800
13	AAPL	Sell	NASDAQ	251.9000	600	0.0000	0.0000
14	002594	Sell	SZSE	101.6500	4000	203.3000	0.0000
15	00700	Sell	HKEX	489.2000	200	97.8400	0.0000
16	00941	Sell	HKEX	80.0500	1000	80.0500	0.0000
17	601988	Sell	SSE	5.9400	5000	14.8500	0.0000
18	NVDA	Sell	NASDAQ	177.3900	200	0.0000	0.0000
19	JPM	Sell	NYSE	294.6000	100	0.0000	35.7000
20	01211	Sell	HKEX	103.9000	1000	103.9000	0.0000

Query 3 SQL: Total Realized P&L by Portfolio and Currency

```sql
SELECT
    s.TradeCurrency AS Currency,
    p.Name AS Portfolio_Name,
    ROUND(SUM(
        (
            tr_sell.ExecutionPrice - (
                SELECT t_b.ExecutionPrice
                FROM TradeRecord t_b
                WHERE t_b.Stock_Code = tr_sell.Stock_Code
                  AND t_b.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                  AND t_b.Direction = 'Buy'
                  AND (
                        SELECT SUM(tb2.Volume)
                        FROM TradeRecord tb2
                        WHERE tb2.Direction = 'Buy'
                          AND tb2.Stock_Code = tr_sell.Stock_Code
                          AND tb2.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                          AND (
                                tb2.Time < t_b.Time
                                OR (tb2.Time = t_b.Time AND tb2.TradeID <= t_b.TradeID)
                              )
                      ) >= (
                        SELECT SUM(ts2.Volume)
                        FROM TradeRecord ts2
                        WHERE ts2.Direction = 'Sell'
                          AND ts2.Stock_Code = tr_sell.Stock_Code
                          AND ts2.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                          AND (
                                ts2.Time < tr_sell.Time
                                OR (ts2.Time = tr_sell.Time AND ts2.TradeID <= tr_sell.TradeID)
                              )
                      )
                ORDER BY t_b.Time ASC, t_b.TradeID ASC
                LIMIT 1
            )
        ) * tr_sell.Volume
        - tr_sell.StampDuty
        - tr_sell.CapitalGainTax
    ), 2) AS Total_Realized_PNL
FROM TradeRecord tr_sell
JOIN Stock s ON tr_sell.Stock_Code = s.Code
JOIN Portfolio p ON tr_sell.Portfolio_PortfolioID = p.PortfolioID
WHERE tr_sell.Direction = 'Sell'
  AND tr_sell.Portfolio_PortfolioID = 1
GROUP BY s.TradeCurrency, p.Name;
```

Query 3 Result: Total Realized P&L by Portfolio and Currency

- The query aggregates realized profit for sell trades after stamp duty and capital gain tax.
- `Main_Aggressive` shows total realized P&L of 2067.75 in USD.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Currency	Portfolio_Name	Total_Realized_PNL
▶	USD	Main_Aggressive	-2067.75

Query 4 SQL (Part 1): Transaction-Level Realized Profit Detail

```sql
SELECT
    p.PortfolioID AS PID,
    p.Name AS Portfolio_Name,
    tr_sell.Time AS Transaction_Time,
    tr_sell.Stock_Code AS Company_Code,
    c.Name AS Company_Name,
    e.Name AS Exchange,
    s.TradeCurrency AS Currency,
    tr_sell.Volume,
    (
        SELECT tr_b.ExecutionPrice
        FROM TradeRecord tr_b
        WHERE tr_b.Stock_Code = tr_sell.Stock_Code
          AND tr_b.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
          AND tr_b.Direction = 'Buy'
          AND (
                SELECT SUM(tb2.Volume)
                FROM TradeRecord tb2
                WHERE tb2.Direction = 'Buy'
                  AND tb2.Stock_Code = tr_sell.Stock_Code
                  AND tb2.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                  AND (
                        tb2.Time < tr_b.Time
                        OR (tb2.Time = tr_b.Time AND tb2.TradeID <= tr_b.TradeID)
                      )
              ) >= (
                SELECT SUM(ts2.Volume)
                FROM TradeRecord ts2
                WHERE ts2.Direction = 'Sell'
                  AND ts2.Stock_Code = tr_sell.Stock_Code
                  AND ts2.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                  AND (
                        ts2.Time < tr_sell.Time
                        OR (ts2.Time = tr_sell.Time AND ts2.TradeID <= tr_sell.TradeID)
                      )
              )
        ORDER BY tr_b.Time ASC, tr_b.TradeID ASC
        LIMIT 1
    ) AS Matching_Buy_Price,
    tr_sell.ExecutionPrice AS Selling_Price,
    tr_sell.StampDuty,
    tr_sell.CapitalGainTax,
```

Query 4 SQL (Part 2): Transaction-Level Realized Profit Detail

```sql
    ROUND(
        (
            tr_sell.ExecutionPrice - (
                SELECT tr_b.ExecutionPrice
                FROM TradeRecord tr_b
                WHERE tr_b.Stock_Code = tr_sell.Stock_Code
                  AND tr_b.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                  AND tr_b.Direction = 'Buy'
                  AND (
                        SELECT SUM(tb2.Volume)
                        FROM TradeRecord tb2
                        WHERE tb2.Direction = 'Buy'
                          AND tb2.Stock_Code = tr_sell.Stock_Code
                          AND tb2.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                          AND (
                                tb2.Time < tr_b.Time
                                OR (tb2.Time = tr_b.Time AND tb2.TradeID <= tr_b.TradeID)
                              )
                      ) >= (
                        SELECT SUM(ts2.Volume)
                        FROM TradeRecord ts2
                        WHERE ts2.Direction = 'Sell'
                          AND ts2.Stock_Code = tr_sell.Stock_Code
                          AND ts2.Portfolio_PortfolioID = tr_sell.Portfolio_PortfolioID
                          AND (
                                ts2.Time < tr_sell.Time
                                OR (ts2.Time = tr_sell.Time AND ts2.TradeID <= tr_sell.TradeID)
                              )
                      )
                ORDER BY tr_b.Time ASC, tr_b.TradeID ASC
                LIMIT 1
            )
        ) * tr_sell.Volume
        - tr_sell.StampDuty
        - tr_sell.CapitalGainTax
    , 2) AS Net_Realized_Profit
FROM TradeRecord tr_sell
JOIN Portfolio p ON tr_sell.Portfolio_PortfolioID = p.PortfolioID
JOIN Stock s ON tr_sell.Stock_Code = s.Code
JOIN Company c ON s.Company_Name = c.Name
JOIN Exchange e ON s.Exchange_Code = e.Code
WHERE tr_sell.Direction = 'Sell'
  AND p.PortfolioID = 1
ORDER BY s.TradeCurrency, tr_sell.Time, tr_sell.TradeID;
```

Query 4 Result: Transaction-Level Realized Profit Detail

- The detail query shows each sell transaction together with its FIFO buy match and tax deductions.
- Portfolio 1 contains one NVDA gain, one AAPL loss, and one break-even NVDA exit.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PID	Portfolio_Name	Transaction_Time	Company_Code	Company_Name	Exchange	Currency	Volume	Matching_Buy_Price	Selling_Price	StampDuty	CapitalGainTax	Net_Realized_Prof
1	Main_Aggressive	2026-04-02 13:30:00	NVDA	NVIDIA Corp	NASDAQ Market	USD	500	175.8600	177.3900	0.0000	114.7500	650.25
1	Main_Aggressive	2026-04-02 14:15:00	AAPL	Apple Inc.	NASDAQ Market	USD	600	256.4300	251.9000	0.0000	0.0000	-2718.00
1	Main_Aggressive	2026-04-02 15:30:00	NVDA	NVIDIA Corp	NASDAQ Market	USD	200	177.3900	177.3900	0.0000	0.0000	0.00

Query 5 SQL: Negative-Growth Companies with PE Ratio

```sql
SELECT
    c.Name AS Company_Name,
    s.Code AS Stock_Code,
    s.TradeCurrency AS Currency,
    e.Name AS Exchange_Name,
    s.Industry,
    c.NetProfitGrowth AS Growth_Rate,
    CASE
        WHEN md.EPS IS NOT NULL AND md.EPS <> 0
            THEN ROUND(md.Close / md.EPS, 2)
        ELSE NULL
    END AS PE_Ratio
FROM Stock s
JOIN Company c ON s.Company_Name = c.Name
JOIN Exchange e ON s.Exchange_Code = e.Code
JOIN MarketData md ON s.Code = md.Stock_Code
WHERE c.NetProfitGrowth < 0
  AND DATE(md.Date) = '2026-04-02'
ORDER BY c.NetProfitGrowth ASC;
```

Query 5 Result: Negative-Growth Companies with PE Ratio

- The screening query joins company, stock, exchange, and market data, then filters for negative profit growth.
- The output includes Chagee, BYD, JPMorgan Chase, and China Mobile with computed PE ratios.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Company_Name	Stock_Code	Currency	Exchange_Name	Industry	Growth_Rate	PE_Ratio
Chagee Holdings	CHA	USD	NASDAQ Market	Consumer Disc	-0.5000	11.55
BYD Company	002594	CNY	Shenzhen Stock Exchange	Automotive	-0.1900	28.39
BYD Company	01211	HKD	HK Exchanges and Clearing	Automotive	-0.1900	26.23
JPMorgan Chase	JPM	USD	New York Stock Exchange	Banking	-0.0200	13.93
China Mobile	00941	HKD	HK Exchanges and Clearing	Telecom	-0.0100	11.42
China Mobile	600941	CNY	Shanghai Stock Exchange	Telecom	-0.0100	14.73

Query 6 SQL: Portfolios with Repeated Sell Activity

```sql
SELECT
    p.PortfolioID,
    p.Name AS Portfolio_Name,
    s.TradeCurrency,
    COUNT(*) AS Num_Sell_Trades,
    ROUND(SUM((tr.ExecutionPrice * tr.Volume) - tr.StampDuty - tr.CapitalGainTax), 2) AS Total_Sell_Amount_After_Tax
FROM TradeRecord tr
JOIN Portfolio p ON tr.Portfolio_PortfolioID = p.PortfolioID
JOIN Stock s ON tr.Stock_Code = s.Code
WHERE tr.Direction = 'Sell'
GROUP BY p.PortfolioID, p.Name, s.TradeCurrency
HAVING COUNT(*) >= 2;
```

Query 6 Result: Portfolios with Repeated Sell Activity

- The query groups sell trades by portfolio and currency, then keeps only groups with at least two sells.
- Three portfolio-currency groups qualify, including `Main_Aggressive` in USD.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PortfolioID	Portfolio_Name	TradeCurrency	Num_Sell_Trades	Total_Sell_Amount_After_Tax
1	Main_Aggressive	USD	3	275198.25
2	Retirement_Fund	USD	2	59788.72
4	Dividend_Growth	HKD	2	177712.11

Technical Highlights

- Primary-key and foreign-key structure organizes the schema around clean links between portfolios, trades, stocks, exchanges, strategies, factors, and market data.
- Multi-table joins support realistic business reporting instead of isolated table lookups.
- Nested subqueries and a CTE with cumulative logic are used to model FIFO cost basis for sell transactions.
- Query design is business-oriented: taxes are exchange specific, realized profit is portfolio specific, and screening logic uses both fundamentals and market metrics.
- The overall logic moves from raw records to decision-ready outputs: base tables store data, update queries calculate tax fields, nested queries resolve FIFO matching, and summary queries transform those results into reports.
- The project reflects real trade-analysis ideas such as stamp duty, capital gain tax, FIFO matching, valuation ratios, and realized profit measurement.

Bonus Prototype Demo

- We completed the bonus task by building a simple local Flask application connected to the MySQL trade database.
- The implementation uses:
  - Flask for routing and page rendering
  - PyMySQL for direct database access
  - HTML templates for dashboard, company browser, trade CRUD, reports, and connection settings
  - SQL queries that reuse the project tables instead of mock data
- The prototype is local only, with no authentication and no deployment, which matches the project bonus description.
- This extension shows that the database is not only well designed in SQL, but also usable in a small application workflow.

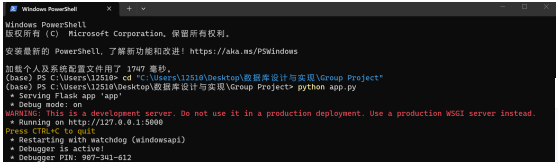
Prototype Implementation Logic

- The prototype follows the same logic as the database project and turns the schema into a small usable application.
- Step 1: start the Flask app locally in PowerShell with `python app.py`.
- Step 2: open the connection settings page and enter host, port, username, password, target schema, and SSL option.
- Step 3: after the connection test succeeds, the app runs SQL against the actual MySQL database and displays live results.
- Step 4: the interface supports browsing reference data, viewing summarized dashboard metrics, reading query-based reports, and editing `TradeRecord` rows.
- This design keeps the architecture simple: browser $\rightarrow$ Flask routes $\rightarrow$ PyMySQL queries $\rightarrow$ MySQL database.

Prototype Setup and Secure Connection

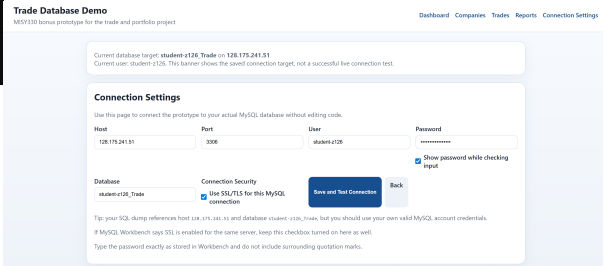
Local startup

- The Flask server is launched from PowerShell inside the project folder.
- The app runs locally at `http://127.0.0.1:5000`.
- This keeps the demo simple and classroom ready.



Database connection

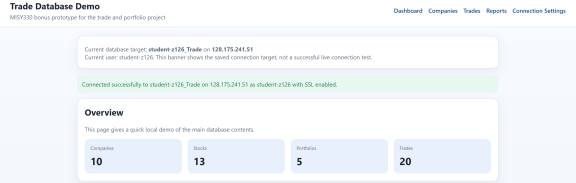
- A settings page lets us configure the real MySQL target without editing code.
- The connection uses the course server, project schema, user credentials, and SSL/TLS because Workbench confirmed secure access.
- This page also helped us debug connection issues step by step during implementation.



Prototype Running Result

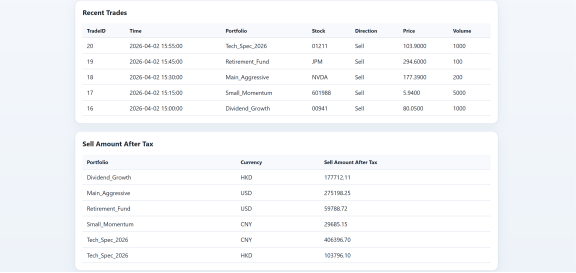
Connection success and dashboard

- After a successful test, the app confirms the database target and SSL status.
- The dashboard immediately shows live counts for companies, stocks, portfolios, and trades.
- This proves the Flask app is reading real data from the database.



Data display and reporting

- The dashboard continues with recent trades and a sell-amount-after-tax summary.
- Other pages also support company browsing, trade CRUD, and query-based reports.
- Together, the prototype demonstrates a complete local front-end workflow on top of the project database.



Recent Trades

TradeID	Time	Portfolio	Stock	Direction	Price	Volume
20	2026-04-02 15:55:00	Tech_Spec_2026	01211	Sell	103.9000	1000
19	2026-04-02 15:45:00	Retirement_Fund	JPM	Sell	294.6000	100
18	2026-04-02 15:30:00	Main_Aggressive	NVDA	Sell	177.3900	200
17	2026-04-02 15:15:00	Small_Momentum	601988	Sell	5.9400	5000
16	2026-04-02 15:00:00	Dividend_Growth	00941	Sell	80.0500	1000

Sell Amount After Tax

Portfolio	Currency	Sell Amount After Tax
Dividend_Growth	HKD	177712.11
Main_Aggressive	USD	275198.25
Retirement_Fund	USD	59788.72
Small_Momentum	CNY	29685.15
Tech_Spec_2026	CNY	406396.70
Tech_Spec_2026	HKD	103796.10

Challenges and Improvements

Challenges

- Designing a schema that connects market data, strategy rules, and trade records without redundancy
- Implementing FIFO-related logic with SQL subqueries and trade ordering
- Handling exchange-specific tax rules across HK, mainland China, and US markets
- Maintaining consistent key values across all sample records

Future Improvements

- Add views for reusable reporting outputs
- Add stored procedures or functions for tax and FIFO calculation
- Extend the Flask prototype with more pages, filters, and form validation
- Add authentication and user-role control if the prototype is expanded beyond a classroom demo
- Extend market data history for richer performance analytics

Current version already includes a working local Flask prototype, but it is still intentionally lightweight and focused on classroom demonstration.

Conclusion

- This database integrates business reference data, market data, strategy rules, and transaction records into one consistent trade-analysis system.
- The project supports meaningful business tasks such as tax calculation, FIFO cost matching, stock screening, and portfolio profit reporting.
- Overall, the system demonstrates how database design and SQL implementation can support practical portfolio trading analysis.

Contribution and AI Usage

- **Yingqi Tan:** business background, presentation organization, and final review.
- **Zhesheng Xu:** schema design, ER/logical modeling, and MySQL implementation.
- **Hao Yang:** SQL query design, result verification, and slide integration.
- **Shared contribution:** all members discussed the database logic, reviewed the project outputs, and understood the complete workflow.
- **AI usage statement:** AI tools were used appropriately for debugging, wording refinement, and presentation drafting support; the database design, SQL logic, and final verification were completed and checked by the group.

Any Questions?

Thank you for listening.